LAPORAN PRAKTIKUM

Mata Kuliah: Otomata

Departemen Teknik Informatika

Institut Teknologi Sepuluh Nopember

TUGAS PRAKTIKUM #1

Lexical Analyzer

Tokenizer untuk Analisis Leksikal Program Komputer

Mata Kuliah : Otomata

Semester : Genap 2025/2026

Kelompok : B03

No	Nama	NRP
1	Agil Lukman Hakim Muchdi	5025241037
2	Berwyn Rafif Alvaro	5025241029
3	Mahendra Agung Darmawan	5025241032

Surabaya, 2026

Daftar Isi

1 Pendahuluan **3**
1.1 Latar Belakang . . . 3
1.2 Tujuan Praktikum . . . 3
1.3 Ruang Lingkup . . . 3

2 Dasar Teori **4**
2.1 Analisis Leksikal . . . 4
2.2 Token . . . 4
2.3 Kategori Token . . . 4
2.4 Regular Expression dalam Tokenisasi . . . 4

3 Perancangan Program **5**
3.1 Arsitektur Sistem . . . 5
3.2 Definisi Token . . . 5
3.2.1 Reserve Words . . . 5
3.2.2 Simbol dan Operator . . . 5
3.3 Algoritma Tokenisasi . . . 6
3.3.1 Pola Regular Expression . . . 6
3.3.2 Alur Klasifikasi Token . . . 7
3.4 Antarmuka Pengguna . . . 7

4 Implementasi **7**
4.1 Lingkungan Pengembangan . . . 7
4.2 Struktur Kode . . . 8
4.3 Fitur yang Diimplementasikan . . . 8

5 Hasil dan Pengujian **9**
5.1 Input Pengujian: Program C++ . . . 9
5.2 Hasil Tokenisasi . . . 9
5.2.1 Reserve Words . . . 9
5.2.2 Variabel (Identifier) . . . 10
5.2.3 Kalimat Matematika . . . 10
5.3 Statistik Distribusi Token . . . 10

6 Pembahasan **11**
6.1 Keunggulan Implementasi . . . 11
6.2 Keterbatasan . . . 11
6.3 Kompleksitas Algoritma . . . 12

7 Kesimpulan **12**

Daftar Pustaka **13**

A Tautan Google Colab **14**

B Source Code Lengkap **14**

1. Pendahuluan

1.1 Latar Belakang

Analisis leksikal merupakan tahap pertama dalam proses kompilasi sebuah program komputer. Pada tahap ini, rangkaian karakter dari *source code* dipecah menjadi satuan-satuan yang bermakna yang disebut *token*. Proses ini dilakukan oleh komponen yang disebut *lexer* atau *tokenizer*.

Pemahaman terhadap proses analisis leksikal sangat penting dalam studi Teori Bahasa dan Automata, karena konsep ini menjadi fondasi dalam perancangan kompiler dan interpreter bahasa pemrograman modern. Setiap token yang dihasilkan akan menjadi masukan bagi tahap analisis sintaksis berikutnya.

1.2 Tujuan Praktikum

Berdasarkan soal Tugas Praktikum #1, tujuan dari praktikum ini adalah:

1. Membangun program komputer yang dapat membaca inputan berupa *source code* program lain.
2. Menghasilkan output berupa token-token (string-string yang terbaca) beserta kategorisasinya.
3. Mengelompokkan token-token tersebut sesuai dengan sifatnya, yaitu:
   a. **Reserve words** – kata kunci yang sudah didefinisikan oleh bahasa pemrograman.
   b. **Simbol dan tanda baca** – karakter spesial yang memiliki makna sintaksis.
   c. **Variabel** – *identifier* yang didefinisikan oleh pengguna.
   d. **Kalimat matematika** – ekspresi dan operator matematika beserta literal angka.
4. Merancang *user interface* yang mudah digunakan oleh pengguna.

1.3 Ruang Lingkup

Program yang dikembangkan mampu menganalisis *source code* dari berbagai bahasa pemrograman populer, di antaranya C/C++, Python, dan Java. Selain empat kategori token yang diwajibkan, program juga mengklasifikasikan *string literal* dan komentar sebagai kategori tambahan untuk kelengkapan analisis.

2. Dasar Teori

2.1 Analisis Leksikal

Analisis leksikal (*lexical analysis*) adalah proses membaca karakter-karakter dari *source code* secara berurutan dan mengubahnya menjadi serangkaian token yang bermakna. Dalam konteks kompiler, analisis leksikal merupakan fase pertama dari proses kompilasi. Output dari *lexer* berupa *token stream* yang kemudian diproses oleh *parser* pada fase analisis sintaksis.

2.2 Token

Token adalah satuan terkecil yang bermakna dalam suatu bahasa pemrograman. Setiap token memiliki tiga atribut utama, yaitu: (1) **nilai** (*lexeme*) yang merupakan string aktual dari *source code*, (2) **tipe** (*token type*) yang merupakan kategori dari token tersebut, dan (3) **posisi** yang menunjukkan nomor baris tempat token ditemukan dalam kode sumber.

2.3 Kategori Token

Tabel 1: Kategori Token pada Lexical Analyzer

Kategori	**Deskripsi**	**Contoh**
Reserve Words	Kata kunci yang telah didefinisikan oleh bahasa pemrograman	`if, for, int, class`
Simbol & Tanda Baca	Karakter spesial dengan makna sintaksis	`( ) { } ; ,`
Variabel	Identifier yang didefinisikan oleh pengguna	`x, myVar, counter`
Kalimat Matematika	Operator dan literal angka	`+, ==, 3.14, 100`
String Literal	Nilai string yang diapit tanda kutip	`"Hello", 'World'`
Komentar	Bagian kode yang tidak dieksekusi	`// ..., /* ... */`

2.4 Regular Expression dalam Tokenisasi

Proses tokenisasi umumnya memanfaatkan *regular expression* (regex) untuk mencocokkan pola-pola token dalam *source code*. Python menyediakan modul `re` yang memungkinkan pencocokan pola secara efisien menggunakan metode `finditer()`. Dengan *named groups* pada regex, setiap kelompok pola dapat diberi nama sehingga klasifikasi token menjadi lebih mudah dan terstruktur.

3. Perancangan Program

3.1 Arsitektur Sistem

Program Lexical Analyzer dirancang dengan arsitektur modular yang terdiri dari empat komponen utama, yaitu: (1) **Definisi Token**, berupa kamus dan set yang mendefinisikan *reserve words*, simbol, dan operator; (2) **Tokenizer Engine**, yaitu fungsi utama yang melakukan proses tokenisasi menggunakan regex; (3) **HTML Generator**, yaitu fungsi untuk menghasilkan tampilan tabel dan statistik; serta (4) **User Interface**, yaitu antarmuka interaktif menggunakan library `ipywidgets`.

3.2 Definisi Token

3.2.1 Reserve Words

Program mendefinisikan sekumpulan *reserve words* yang mencakup kata kunci dari berbagai bahasa pemrograman (C/C++, Python, Java):

```
RESERVE_WORDS = {
    'if', 'else', 'elif', 'while', 'for', 'do', 'switch', 'case',
    'break', 'continue', 'return', 'function', 'func', 'def', 'class',
    'int', 'float', 'double', 'char', 'string', 'bool', 'boolean',
    'void', 'null', 'None', 'True', 'False', 'true', 'false',
    'import', 'from', 'as', 'in', 'not', 'and', 'or', 'is',
    'try', 'except', 'finally', 'raise', 'with', 'pass', 'lambda',
    'new', 'delete', 'this', 'self', 'super', 'static', 'public',
    # ... dan lainnya
}
```

Listing 1: Definisi Reserve Words

3.2.2 Simbol dan Operator

Program menggunakan dua kamus terpisah untuk simbol tanda baca dan operator matematika:

```
SYMBOLS = {
    '(': 'LEFT_PAREN',    ')': 'RIGHT_PAREN',
    '{': 'LEFT_BRACE',    '}': 'RIGHT_BRACE',
    '[': 'LEFT_BRACKET', ']': 'RIGHT_BRACKET',
    ';': 'SEMICOLON',     ':': 'COLON',
```

```
    ',': 'COMMA',          '.': 'DOT',
}

MATH_OPERATORS = {
    '+': 'PLUS',           '-': 'MINUS',
    '*': 'MULTIPLY',       '/': 'DIVIDE',
    '==': 'EQUAL',         '!=': 'NOT_EQUAL',
    '<=': 'LESS_EQUAL',    '>=': 'GREATER_EQUAL',
    '++': 'INCREMENT',     '--': 'DECREMENT',
    '&&': 'LOGICAL_AND',   '||': 'LOGICAL_OR',
    # ... dan lainnya
}
```

Listing 2: Definisi Simbol dan Operator

3.3 Algoritma Tokenisasi

3.3.1 Pola Regular Expression

Inti dari tokenizer adalah satu pola regex yang menggabungkan semua kemungkinan token dalam urutan prioritas yang tepat:

```
token_pattern = re.compile(
    r'(?P<COMMENT_ML>/\*[\s\S]*?\*/)'          # Komentar multi-baris
    r'|(?P<COMMENT_SL>//[^\n]*)'                # Komentar satu baris
    r'|(?P<COMMENT_PY>#[^\n]*)'                 # Komentar Python
    r'|(?P<STRING_D>"(?:[^"\\]|\\.)*")'         # String double-quote
    r'|(?P<STRING_S>\'(?:[^\'\\]|\\.)*\')'      # String single-quote
    r'|(?P<FLOAT>\d+\.\d+)'                     # Bilangan desimal
    r'|(?P<INTEGER>\d+)'                        # Bilangan bulat
    r'|(?P<OP_DOUBLE>[=!<>+\-*/&|:]{2})'       # Operator 2 karakter
    r'|(?P<IDENTIFIER>[a-zA-Z_][a-zA-Z0-9_]*)' # Identifier
    r'|(?P<OPERATOR>[+\-*/%^=<>!&|])'           # Operator 1 karakter
    r'|(?P<SYMBOL>[(){}\[\];:,\.#@~?\\])'       # Simbol
    r'|(?P<NEWLINE>\n)'                         # Baris baru
    r'|(?P<WHITESPACE>[ \t]+)'                  # Spasi/tab
    r'|(?P<UNKNOWN>.)'                          # Karakter tak dikenal
)
```

Listing 3: Pola Regular Expression Tokenizer

3.3.2 Alur Klasifikasi Token

Setelah pencocokan regex, setiap token diklasifikasikan berdasarkan *named group* yang berhasil dicocokkan. Urutan alur klasifikasinya adalah sebagai berikut:

1. `COMMENT_ML / COMMENT_SL / COMMENT_PY` → Komentar
2. `STRING_D / STRING_S` → String Literal
3. `FLOAT / INTEGER` → Kalimat Matematika (Number)
4. `IDENTIFIER` ∈ `RESERVE_WORDS` → Reserve Word
5. `IDENTIFIER` ∉ `RESERVE_WORDS` → Variabel
6. `OP_DOUBLE / OPERATOR` → Kalimat Matematika (Operator)
7. `SYMBOL` → Simbol dan Tanda Baca
8. `NEWLINE / WHITESPACE` → (diabaikan, hanya untuk pelacakan baris)
9. `UNKNOWN` → Token tidak dikenal

3.4 Antarmuka Pengguna

Antarmuka program dibangun menggunakan library `ipywidgets` yang menyediakan komponen interaktif di Jupyter Notebook dan Google Colab. Komponen yang disediakan meliputi: area teks untuk memasukkan *source code*, *dropdown* untuk memilih contoh program, tombol analisis, tombol *clear*, tampilan hasil per kategori menggunakan *accordion*, serta panel statistik distribusi token.

4. Implementasi

4.1 Lingkungan Pengembangan

Tabel 2: Lingkungan Pengembangan

Komponen	**Keterangan**
Bahasa Pemrograman	Python 3.x
Platform	Jupyter Notebook / Google Colab
Library Utama	`re`, `ipywidgets`, `IPython.display`
Instalasi Tambahan	`pip install ipywidgets`

4.2 Struktur Kode

Program terdiri dari tujuh bagian utama yang disusun secara berurutan:

1. **Instalasi dan Import** – instalasi library dan impor modul yang diperlukan.
2. **Definisi Token** – set dan kamus untuk *reserve words*, simbol, dan operator.
3. **Fungsi Tokenizer** – fungsi `tokenize()` sebagai inti analisis leksikal.
4. **Fungsi HTML Generator** – fungsi `get_color()`, `generate_table_html()`, dan `generate_stats_html()`.
5. **Sample Codes** – contoh program C++, Python, Java, dan ekspresi matematika.
6. **UI Builder** – fungsi `build_ui()` untuk antarmuka interaktif.
7. **Entry Point** – pemanggilan `build_ui()` untuk menjalankan program.

4.3 Fitur yang Diimplementasikan

Selain empat kategori token wajib, program dilengkapi dengan fitur-fitur berikut:

- Dukungan multi-bahasa pemrograman (C/C++, Python, Java).
- Pelacakan nomor baris untuk setiap token yang ditemukan.
- Panel statistik distribusi token dalam bentuk grafik batang dan kartu ringkasan.
- Tabel frekuensi 10 token yang paling sering muncul.
- Empat contoh *source code* siap pakai untuk pengujian cepat.
- Penanganan operator dua karakter seperti `==`, `!=`, `<=`, `++`.
- Penanganan komentar multi-baris `/* ... */`.

5. Hasil dan Pengujian

5.1 Input Pengujian: Program C++

Berikut adalah contoh *source code* C++ yang digunakan sebagai input pengujian:

```
// Program C++ Sederhana
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    float pi = 3.14159;
    string name = "Hello World";

    if (x > 5 && pi < 4.0) {
        x += 1;
        pi *= 2;
    }

    for (int i = 0; i < x; i++) {
        cout << i << endl;
    }

    return 0;
}
```

Listing 4: Input Pengujian: Program C++ Sederhana

5.2 Hasil Tokenisasi

5.2.1 Reserve Words

Tabel 3: Daftar Reserve Words yang Teridentifikasi

No	Token	Sub-Tipe	Baris
1	`include`	KEYWORD	2
2	`using`	KEYWORD	3
3	`namespace`	KEYWORD	3
4	`int`	KEYWORD	5
5	`int`	KEYWORD	6
6	`float`	KEYWORD	7
7	`string`	KEYWORD	8
8	`if`	KEYWORD	10
9	`for`	KEYWORD	15
10	`int`	KEYWORD	15
11	`return`	KEYWORD	19

5.2.2 Variabel (Identifier)

Tabel 4: Daftar Variabel yang Teridentifikasi

No	Token	Sub-Tipe	Baris
1	`iostream`	IDENTIFIER	2
2	`std`	IDENTIFIER	3
3	`main`	IDENTIFIER	5
4	`x`	IDENTIFIER	6
5	`pi`	IDENTIFIER	7
6	`name`	IDENTIFIER	8
7	`i`	IDENTIFIER	15
8	`cout`	IDENTIFIER	16
9	`endl`	IDENTIFIER	16

5.2.3 Kalimat Matematika

Tabel 5: Daftar Kalimat Matematika yang Teridentifikasi

No	Token	Sub-Tipe	Baris
1	`10`	Number (INTEGER)	6
2	`3.14159`	Number (FLOAT)	7
3	`>`	Operator (GREATER_THAN)	10
4	`&&`	Operator (LOGICAL_AND)	10
5	`<`	Operator (LESS_THAN)	10
6	`4.0`	Number (FLOAT)	10
7	`+=`	Operator (PLUS_ASSIGN)	11
8	`1`	Number (INTEGER)	11
9	`*=`	Operator (MULTIPLY_ASSIGN)	12
10	`2`	Number (INTEGER)	12
11	`++`	Operator (INCREMENT)	15
12	`«`	Operator (LEFT_SHIFT)	16

5.3 Statistik Distribusi Token

Dari pengujian terhadap program C++ di atas, diperoleh distribusi token sebagai berikut:

Tabel 6: Distribusi Token pada Program C++

Kategori	**Jumlah**	**Persentase (%)**
Reserve Words	11	13,9
Simbol dan Tanda Baca	24	30,4
Variabel	19	24,1
Kalimat Matematika	16	20,3
String Literal	1	1,3
Komentar	1	1,3
Lainnya	7	8,9
Total	**79**	**100**

6. Pembahasan

6.1 Keunggulan Implementasi

Beberapa keunggulan dari implementasi yang dikembangkan antara lain:

1. **Regex Terpadu:** Seluruh pola token digabungkan dalam satu ekspresi reguler menggunakan *named groups*, sehingga pemrosesan lebih efisien dibandingkan pendekatan iteratif.

2. **Prioritas Pencocokan:** Pola disusun dari yang paling spesifik ke yang paling umum. Misalnya, operator dua karakter (`OP_DOUBLE`) dicocokkan sebelum operator satu karakter (`OPERATOR`), sehingga `==` tidak salah dikenali sebagai dua buah `=`.

3. **Dukungan Multi-bahasa:** *Reserve words* mencakup kata kunci dari C/C++, Python, dan Java, sehingga program dapat menganalisis berbagai bahasa pemrograman.

4. **Pelacakan Nomor Baris:** Nomor baris dilacak dengan menghitung token `NEWLINE`, sehingga output dapat digunakan untuk keperluan *debugging*.

6.2 Keterbatasan

Terdapat beberapa keterbatasan pada implementasi yang dikembangkan:

1. *Preprocessor directive* C/C++ seperti `#include` diproses dengan memisahkan karakter `#` sebagai simbol dan `include` sebagai *identifier*, bukan sebagai satu kesatuan token.

2. Program tidak melakukan analisis sintaksis, sehingga tidak dapat mendeteksi kesalahan sintaks pada *source code* yang dimasukkan.

3. Beberapa kata seperti `cout` dan `endl` yang sebenarnya merupakan bagian dari pustaka standar C++ turut dimasukkan ke dalam daftar *reserve words*.

6.3 Kompleksitas Algoritma

Algoritma tokenisasi menggunakan pendekatan *single-pass* dengan kompleksitas waktu $O(n)$, di mana n adalah panjang *source code*. Hal ini memungkinkan program memproses *source code* dengan cepat tanpa perlu iterasi berulang terhadap masukan yang sama.

7. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Program Lexical Analyzer berhasil diimplementasikan menggunakan Python dengan memanfaatkan modul `re` dan library `ipywidgets`.

2. Program mampu mengklasifikasikan token ke dalam enam kategori, yaitu *reserve words*, simbol dan tanda baca, variabel, kalimat matematika, *string literal*, dan komentar.

3. Antarmuka pengguna yang interaktif berhasil dirancang dengan fitur pemilihan contoh program, navigasi per kategori, dan panel statistik distribusi token.

4. Penggunaan *regular expression* dengan *named groups* terbukti efektif untuk implementasi tokenizer yang efisien dengan kompleksitas $O(n)$.

5. Program mendukung analisis multi-bahasa pemrograman dan mampu menangani konstruksi seperti komentar multi-baris, *string literal*, dan operator majemuk.

Daftar Pustaka

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.

[2] Python Software Foundation. (2024). *re — Regular expression operations*. Diakses dari `https://docs.python.org/3/library/re.html`

[3] Project Jupyter. (2024). *ipywidgets — Interactive HTML Widgets*. Diakses dari `https://ipywidgets.readthedocs.io/`

[4] Grune, D., & Jacobs, C. J. H. (2008). *Parsing Techniques: A Practical Guide* (2nd ed.). Springer.

[5] Sebesta, R. W. (2016). *Concepts of Programming Languages* (11th ed.). Pearson.

LAMPIRAN A. Tautan Google Colab

Program Lexical Analyzer ini dijalankan menggunakan Google Colaboratory (Google Colab) agar dapat diakses dan dieksekusi secara langsung melalui browser tanpa instalasi lokal. Tautan notebook dapat diakses pada alamat berikut:

Platform	Google Colaboratory
Tautan	`https://colab.research.google.com/drive/1G9y5XfwuSl8nz1AASZBCNDT7I2XkTGbs?usp=sharing`
Cara Akses	Buka tautan → File → Open Notebook → Run All

Untuk menjalankan program, cukup buka notebook di Google Colab, kemudian pilih **Runtime** → **Run All** atau jalankan setiap sel secara berurutan dari atas ke bawah. Pastikan koneksi internet tersedia agar library `ipywidgets` dapat diaktifkan dengan benar.

LAMPIRAN B. Source Code Lengkap

Berikut adalah keseluruhan source code program Lexical Analyzer yang diimplementasikan pada Google Colab.

Bagian 1: Instalasi dan Import Library

```
!pip install ipywidgets -q

import re
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output
```

Listing 5: Instalasi dan Import

Bagian 2: Definisi Token

```
RESERVE_WORDS = {
    'if', 'else', 'elif', 'while', 'for', 'do', 'switch', 'case',
    'break', 'continue', 'return', 'function', 'func', 'def', 'class',
    'int', 'float', 'double', 'char', 'string', 'bool', 'boolean',
```

```
    'void', 'null', 'None', 'True', 'False', 'true', 'false',
    'import', 'from', 'as', 'in', 'not', 'and', 'or', 'is',
    'try', 'except', 'finally', 'raise', 'with', 'pass', 'lambda',
    'new', 'delete', 'this', 'self', 'super', 'static', 'public',
    'private', 'protected', 'const', 'var', 'let', 'print', 'input',
    'include', 'using', 'namespace', 'struct', 'enum', 'typedef',
    'sizeof', 'extern', 'volatile', 'register', 'goto', 'default',
    'cout', 'cin', 'endl', 'printf', 'scanf', 'malloc', 'free',
    'long', 'short', 'unsigned', 'signed', 'auto', 'inline'
}

SYMBOLS = {
    '(': 'LEFT_PAREN',    ')': 'RIGHT_PAREN',
    '{': 'LEFT_BRACE',    '}': 'RIGHT_BRACE',
    '[': 'LEFT_BRACKET', ']': 'RIGHT_BRACKET',
    ';': 'SEMICOLON',     ':': 'COLON',
    ',': 'COMMA',         '.': 'DOT',
    '#': 'HASH',          '@': 'AT',
    '~': 'TILDE',         '?': 'QUESTION_MARK',
    '\\': 'BACKSLASH',
}

MATH_OPERATORS = {
    '+': 'PLUS',     '-': 'MINUS',    '*': 'MULTIPLY',
    '/': 'DIVIDE',   '%': 'MODULO',   '^': 'POWER',
    '=': 'ASSIGN',   '<': 'LESS_THAN', '>': 'GREATER_THAN',
    '==': 'EQUAL',       '!=': 'NOT_EQUAL',
    '<=': 'LESS_EQUAL', '>=': 'GREATER_EQUAL',
    '+=': 'PLUS_ASSIGN',     '-=': 'MINUS_ASSIGN',
    '*=': 'MULTIPLY_ASSIGN', '/=': 'DIVIDE_ASSIGN',
    '++': 'INCREMENT',   '--': 'DECREMENT',
    '**': 'POWER',       '//': 'FLOOR_DIVIDE',
    '&&': 'LOGICAL_AND', '||': 'LOGICAL_OR',
    '<<': 'LEFT_SHIFT',  '>>': 'RIGHT_SHIFT',
    '->': 'ARROW',       '::': 'SCOPE',
}
```

Listing 6: Definisi Reserve Words

Bagian 3: Fungsi Tokenizer

```
def tokenize(source_code):
    tokens = {
        'reserve_words': [], 'symbols': [],
        'variables': [],      'math_expressions': [],
        'strings': [],        'comments': [],
        'unknown': []
    }
    all_tokens = []

    token_pattern = re.compile(
        r'(?P<COMMENT_ML>/\*[\s\S]*?\*/)'
        r'|(?P<COMMENT_SL>//[^\n]*)'
        r'|(?P<COMMENT_PY>#[^\n]*)'
        r'|(?P<STRING_D>"(?:[^"\\]|\\.)*")'
        r'|(?P<STRING_S>\'(?:[^\'\\]|\\.)*\')'
        r'|(?P<FLOAT>\d+\.\d+)'
        r'|(?P<INTEGER>\d+)'
        r'|(?P<OP_DOUBLE>[=!<>+\-*/&|:]{2})'
        r'|(?P<IDENTIFIER>[a-zA-Z_][a-zA-Z0-9_]*)'
        r'|(?P<OPERATOR>[+\-*/%^=<>!&|])'
        r'|(?P<SYMBOL>[(){}\[\];:,\.#@~?\])'
        r'|(?P<NEWLINE>\n)'
        r'|(?P<WHITESPACE>[ \t]+)'
        r'|(?P<UNKNOWN>.)'
    )

    line_number = 1

    for match in token_pattern.finditer(source_code):
        kind = match.lastgroup
        value = match.group()

        if kind == 'NEWLINE':
            line_number += 1
            continue
        elif kind == 'WHITESPACE':
            continue
        elif kind in ('COMMENT_ML', 'COMMENT_SL', 'COMMENT_PY'):
            token_info = {'token': value.strip(), 'type': 'Komentar',
                          'subtype': kind, 'line': line_number}
            tokens['comments'].append(token_info)
```

```
        elif kind in ('STRING_D', 'STRING_S'):
            token_info = {'token': value, 'type': 'String Literal',
                          'subtype': kind, 'line': line_number}
            tokens['strings'].append(token_info)
        elif kind in ('FLOAT', 'INTEGER'):
            token_info = {'token': value, 'type': 'Kalimat Matematika',
                          'subtype': f'Number ({kind})', 'line': line_number}
            tokens['math_expressions'].append(token_info)
        elif kind == 'IDENTIFIER':
            if value in RESERVE_WORDS:
                token_info = {'token': value, 'type': 'Reserve Word',
                              'subtype': 'KEYWORD', 'line': line_number}
                tokens['reserve_words'].append(token_info)
            else:
                token_info = {'token': value, 'type': 'Variabel',
                              'subtype': 'IDENTIFIER', 'line': line_number}
                tokens['variables'].append(token_info)
        elif kind in ('OP_DOUBLE', 'OPERATOR'):
            op_name = MATH_OPERATORS.get(value, f'OP_{value}')
            token_info = {'token': value, 'type': 'Kalimat Matematika',
                          'subtype': f'Operator ({op_name})',
                          'line': line_number}
            tokens['math_expressions'].append(token_info)
        elif kind == 'SYMBOL':
            sym_name = SYMBOLS.get(value, f'SYMBOL_{value}')
            token_info = {'token': value, 'type': 'Simbol dan Tanda Baca',
                          'subtype': sym_name, 'line': line_number}
            tokens['symbols'].append(token_info)
        else:
            token_info = {'token': value, 'type': 'Unknown',
                          'subtype': 'UNKNOWN', 'line': line_number}
            tokens['unknown'].append(token_info)
```

```

        all_tokens.append(token_info)

    return tokens, all_tokens
```

Listing 7: Fungsi Tokenize

Bagian 4: Fungsi Generasi Output HTML

```
def get_color(token_type):
    colors = {
        'Reserve Word':           {'bg':'#1a3a5c','text':'#569cd6','badge':'#007acc'},
        'Simbol dan Tanda Baca': {'bg':'#2d2d2d','text':'#d4d4d4','badge':'#555'},
        'Variabel':               {'bg':'#1a3a4a','text':'#9cdcfe','badge':'#0e7490'},
        'Kalimat Matematika':     {'bg':'#1a3a1a','text':'#b5cea8','badge':'#166534'},
        'String Literal':         {'bg':'#3a2a1a','text':'#ce9178','badge':'#92400e'},
        'Komentar':               {'bg':'#1a2a1a','text':'#6a9955','badge':'#14532d'},
        'Unknown':                {'bg':'#3a1a1a','text':'#f44747','badge':'#991b1b'},
    }
    return colors.get(token_type, {'bg':'#2d2d2d','text':'#fff','badge':'#555'})

def generate_table_html(token_list, title="Token"):
    if not token_list:
        return "<div style='text-align:center; padding:30px; color:#666;'>"\
               "Tidak ada token ditemukan untuk kategori ini</div>"

    rows_html = ""
    for i, t in enumerate(token_list, 1):
        c = get_color(t['type'])
        bg = '#1e1e1e' if i % 2 == 0 else '#252525'
        token_escaped = t['token'].replace('<','&lt;').replace('>','&gt;')
        rows_html += f"""
        <tr style='background:{bg};'>
            <td style='padding:8px 12px; color:#666; text-align:center;'>{i}</td>
```

```
            <td style='padding:8px 12px;'>
                <code style='background:{c["bg"]}; color:{c["text"]};
                            padding:2px 8px; border-radius:4px;'>
                    {token_escaped}
                </code>
            </td>
            <td style='padding:8px 12px;'>
                <span style='background:{c["badge"]}; color:white;
                            padding:2px 10px; border-radius:12px;
                            font-size:11px;'>{t['type']}</span>
            </td>
            <td style='padding:8px 12px; color:#888;'>{t['subtype']}</td>
            <td style='padding:8px 12px; color:#666; text-align:center;'>
                {t['line']}
            </td>
        </tr>"""

    return f"""
    <div style='overflow-x:auto; border-radius:8px; border:1px solid #333;'>
        <table style='width:100%; border-collapse:collapse;
                      font-family:Consolas; font-size:13px;'>
            <thead>
                <tr style='background:#2d2d2d;'>
                    <th style='padding:10px 12px; color:#888; width:50px;'>No</th>
                    <th style='padding:10px 12px; color:#ccc; text-align:left;'>Token</th>
                    <th style='padding:10px 12px; color:#ccc; text-align:left;'>Tipe</th>
                    <th style='padding:10px 12px; color:#ccc; text-align:left;'>Sub-Tipe</th>
                    <th style='padding:10px 12px; color:#888; width:70px;'>Baris</th>
                </tr>
            </thead>
            <tbody>{rows_html}</tbody>
        </table>
```

```
    </div>"""
```

Listing 8: Fungsi get_color dan generate_table_html

```
def generate_stats_html(tokens, all_tokens):
    total = len(all_tokens)
    if total == 0:
        return "<p>Tidak ada data</p>"

    categories = [
        ('Reserve Words',          tokens['reserve_words'], '#007acc'),
        ('Simbol dan Tanda Baca', tokens['symbols'], '#555'),
        ('Variabel',               tokens['variables'], '#0e7490'),
        ('Kalimat Matematika',     tokens['math_expressions'], '#166534'),
        ('String Literal',         tokens['strings'], '#92400e'),
        ('Komentar',               tokens['comments'], '#14532d'),
        ('Unknown',                tokens['unknown'], '#991b1b'),
    ]

    bars_html = ""
    for name, token_list, color in categories:
        count = len(token_list)
        pct = (count / total * 100) if total > 0 else 0
        bars_html += f"""
        <div style='margin:8px 0;'>
            <div style='display:flex; justify-content:space-between;'>
                <span style='color:#ccc; font-size:13px;'>{name}</span>
                <span style='color:#888; font-size:12px;'>{count} ({pct:.1f}%)</span>
            </div>
            <div style='background:#333; border-radius:10px; height:10px;'>
                <div style='background:{color}; width:{pct}%;
                            height:100%; border-radius:10px;'></div>
            </div>
        </div>"""
```

```

    token_freq = {}
    for t in all_tokens:
        token_freq[t['token']] = token_freq.get(t['token'], 0) + 1
    sorted_tokens = sorted(token_freq.items(), key=lambda x: x[1], reverse=True)

    freq_rows = ""
    for i, (tok, freq) in enumerate(sorted_tokens[:10], 1):
        pct = freq / total * 100
        freq_rows += f"""
        <tr style='background:{"#1e1e1e" if i%2==0 else "#252525"};'>
            <td style='padding:6px 12px; color:#666; text-align:center;'>{i}</td>
            <td style='padding:6px 12px;'>
                <code style='color:#9cdcfe; background:#1a3a4a;
                            padding:2px 8px; border-radius:4px;'>
                    {tok.replace('<','&lt;').replace('>','&gt;')}
                </code>
            </td>
            <td style='padding:6px 12px; color:#fff; text-align:center;'>{freq}</td>
            <td style='padding:6px 12px; color:#888; text-align:center;'>{pct:.1f}%</td>
        </tr>"""

    return f"""
    <div style='background:#1e1e1e; border-radius:8px; padding:20px;
                margin-bottom:20px; border:1px solid #333;'>
        <h3 style='color:#ccc; margin:0 0 15px 0; font-size:14px;'>
            Distribusi Token
        </h3>
        {bars_html}
    </div>
    <div style='background:#1e1e1e; border-radius:8px; padding:20px;
                border:1px solid #333;'>
        <h3 style='color:#ccc; margin:0 0 15px 0; font-size:14px;'>
```

```
            Top 10 Token Paling Sering Muncul
        </h3>
        <table style='width:100%; border-collapse:collapse;
                      font-family:Consolas; font-size:13px;'
    >
            <thead>
                <tr style='background:#2d2d2d;'>
                    <th style='padding:8px 12px; color:#888;
    '>Rank</th>
                    <th style='padding:8px 12px; color:#ccc;
     text-align:left;'>Token</th>
                    <th style='padding:8px 12px; color:#ccc;
    '>Frekuensi</th>
                    <th style='padding:8px 12px; color:#ccc;
    '>Persentase</th>
                </tr>
            </thead>
            <tbody>{freq_rows}</tbody>
        </table>
    </div>"""
```

Listing 9: Fungsi generate_stats_html

Bagian 5: Sample Codes

```
SAMPLES = {
    'C++ Basic': '''// Program C++ Sederhana
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    float pi = 3.14159;
    string name = "Hello World";
    if (x > 5 && pi < 4.0) { x += 1; pi *= 2; }
    for (int i = 0; i < x; i++) { cout << i << endl; }
    return 0;
}''',

    'Python': '''# Program Python
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

class MyClass:
    def __init__(self, name):
```

```
        self.name = name
    def greet(self):
        return f"Hello, {self.name}"

x = 10
y = 3.14
result = x * y + 2 ** 8
print(result)''',

    'Java': '''// Program Java
public class Main {
    public static void main(String[] args) {
        int x = 100;
        double y = 3.14;
        boolean flag = true;
        String message = "Hello Java";
        if (flag && x > 50) { x -= 10; y *= 2.0; }
        for (int i = 0; i <= 10; i++) {
            System.out.println(i * x);
        }
    }
}''',
}
```

Listing 10: Definisi Sample Codes

Bagian 6: User Interface dan Entry Point

```
def build_ui():
    input_area = widgets.Textarea(
        value=SAMPLES['C++ Basic'],
        placeholder='Masukkan source code di sini...',
        layout=widgets.Layout(width='100%', height='250px')
    )

    sample_dropdown = widgets.Dropdown(
        options=list(SAMPLES.keys()),
        value='C++ Basic',
        description='Sample:',
        layout=widgets.Layout(width='300px')
    )

    btn_analyze = widgets.Button(
        description='ANALISIS TOKEN',
        button_style='',
        layout=widgets.Layout(width='200px', height='40px')
```

```
    )
    btn_clear = widgets.Button(
        description='CLEAR',
        button_style='danger',
        layout=widgets.Layout(width='130px', height='40px')
    )
    btn_load = widgets.Button(
        description='LOAD SAMPLE',
        button_style='success',
        layout=widgets.Layout(width='170px', height='40px')
    )

    btn_row = widgets.HBox(
        [btn_analyze, btn_clear, btn_load, sample_dropdown],
        layout=widgets.Layout(gap='10px', align_items='center', margin='10px 0')
    )
    output = widgets.Output()

    def on_analyze(b):
        with output:
            clear_output(wait=True)
            src = input_area.value
            if not src.strip():
                display(HTML("<p>Masukkan source code terlebih dahulu!</p>"))
                return
            tokens, all_tokens = tokenize(src)
            tabs_data = [
                ("Semua Token",            all_tokens),
                ("Reserve Words",          tokens['reserve_words']),
                ("Simbol & Tanda Baca",    tokens['symbols']),
                ("Variabel",               tokens['variables']),
                ("Kalimat Matematika",     tokens['math_expressions']),
                ("String Literal",         tokens['strings']),
                ("Komentar",               tokens['comments']),
            ]
            accordion_children = []
            titles = []
            for title, token_list in tabs_data:
```

```
                out = widgets.Output()
                with out:
                    display(HTML(generate_table_html(token_list, title)))
                accordion_children.append(out)
                titles.append(f"{title} ({len(token_list)})")
            stat_out = widgets.Output()
            with stat_out:
                display(HTML(generate_stats_html(tokens, all_tokens)))
            accordion_children.append(stat_out)
            titles.append("Statistik")
            accordion = widgets.Accordion(children=accordion_children)
            for i, title in enumerate(titles):
                accordion.set_title(i, title)
            accordion.selected_index = 0
            display(accordion)

    def on_clear(b):
        input_area.value = ''
        with output:
            clear_output()

    def on_load(b):
        input_area.value = SAMPLES[sample_dropdown.value]
        with output:
            clear_output()

    btn_analyze.on_click(on_analyze)
    btn_clear.on_click(on_clear)
    btn_load.on_click(on_load)

    display(input_area, btn_row, output)

# ---- ENTRY POINT ----
build_ui()
```

Listing 11: Fungsi build_ui dan Entry Point